```tsx
import React, { useRef, useEffect, useState, useCallback } from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { Checkbox } from "@/components/selector/checkbox";

// 🚀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🛢 LOCAL TYPE ISOLATION GATEWAY
export interface PopupMenuItemConfig {
  id: string;
  label: string;
  icon?: LucideIcon;
  shortcut?: string;
  disabled?: boolean;
  destructive?: boolean;
  onClick?: () => void;
  children?: PopupMenuItemConfig[]; // Up to 3 layers cascading
  // Toggleable Checkbox capability
  isCheckbox?: boolean;
  checked?: boolean;
  onCheckedChange?: (checked: boolean) => void;
}

export interface PopupMenuProps {
  items: PopupMenuItemConfig[];
  trigger: React.ReactNode;
  triggerType?: "click" | "context-menu";
  className?: string;
}

// Internal types for recursive rendering
interface SubMenuContextValue {
  level: number;
  parentRect?: DOMRect;
  openSubMenus: Set<string>;
  registerSubMenu: (id: string, element: HTMLElement | null) => void;
  unregisterSubMenu: (id: string) => void;
  isSubMenuOpen: (id: string) => boolean;
}

const SubMenuContext = React.createContext<SubMenuContextValue | null>(null);

function useSubMenuContext() {
  const context = React.useContext(SubMenuContext);
  if (!context) {
```

```
    throw new Error("SubMenu components must be rendered within a PopupMenu");
  }
  return context;
}

// Recursive menu item component
interface MenuItemProps {
  item: PopupMenuItemConfig;
  level: number;
  onClose: () => void;
}

function MenuItem({ item, level, onClose }: MenuItemProps) {
  const { openSubMenus, registerSubMenu, unregisterSubMenu, isSubMenuOpen } =
    useSubMenuContext();
  const itemRef = useRef<HTMLDivElement>(null);
  const subMenuRef = useRef<HTMLDivElement>(null);
  const [subMenuPosition, setSubMenuPosition] = useState<{
    top: number;
    left: number;
  } | null>(null);
  const isOpen = isSubMenuOpen(item.id);
  const hasChildren = item.children && item.children.length > 0;
  const isCheckboxItem = item.isCheckbox === true;

  // Register/unregister submenu element
  useEffect(() => {
    registerSubMenu(item.id, subMenuRef.current);
    return () => unregisterSubMenu(item.id);
  }, [item.id, registerSubMenu, unregisterSubMenu]);

  // Calculate submenu position to avoid viewport clipping
  const calculateSubMenuPosition = useCallback(() => {
    if (!itemRef.current) return;

    const rect = itemRef.current.getBoundingClientRect();
    const viewportWidth = window.innerWidth;
    const viewportHeight = window.innerHeight;
    const subMenuWidth = 240; // Approximate submenu width
    const subMenuHeight = 200; // Approximate submenu height

    let left = rect.right;
    let top = rect.top;

    // Check right boundary
    if (left + subMenuWidth > viewportWidth) {
      left = rect.left - subMenuWidth;
    }

    // Check bottom boundary
```

```
  if (top + subMenuHeight > viewportHeight) {
    top = viewportHeight - subMenuHeight;
  }

  // Check top boundary
  if (top < 0) {
    top = 0;
  }

  setSubMenuPosition({ top, left });
}, []);

useEffect(() => {
  if (isOpen) {
    calculateSubMenuPosition();
  }
}, [isOpen, calculateSubMenuPosition]);

const handleKeyDown = (event: React.KeyboardEvent) => {
  switch (event.key) {
    case "Enter":
    case " ":
      event.preventDefault();
      if (!item.disabled && item.onClick) {
        item.onClick();
        onClose();
      }
      break;
    case "ArrowRight":
      event.preventDefault();
      if (hasChildren && !item.disabled) {
        // Open submenu - handled by parent context
      }
      break;
    case "ArrowLeft":
      event.preventDefault();
      // Close submenu - handled by parent context
      break;
  }
};

const handleMouseEnter = () => {
  if (hasChildren && !item.disabled) {
    // Submenu opening is handled by the parent DropdownMenuSub
  }
};

if (item.id === "divider") {
  return (
    <DropdownMenuPrimitive.Separator
```

```
        className="h-px bg-border/50 my-1"
        key={item.id}
      />
    );
  }

  // Render CheckboxItem for toggleable checkbox items
  if (isCheckboxItem) {
    return (
      <DropdownMenuPrimitive.CheckboxItem
        ref={itemRef}
        key={item.id}
        disabled={item.disabled}
        checked={item.checked ?? false}
        onCheckedChange={item.onCheckedChange}
        onSelect={(e) => e.preventDefault()} // ANTI-CLOSE GUARD: Prevent dropdown
from closing on checkbox toggle
        className={cn(
          "relative flex cursor-default select-none items-center rounded-sm px-3
py-2 text-sm outline-none transition-all duration-100",
          "focus:bg-brand-primary/10 focus:text-brand-primary active:scale-[0.98]",
          "data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
          item.destructive
            ? "text-destructive focus:bg-destructive/10 focus:text-destructive"
            : "text-text-main",
          level > 0 && "pl-8",
        )}
        onKeyDown={handleKeyDown}
        onMouseEnter={handleMouseEnter}
      >
        {item.icon && (
          <span className="mr-3 h-4 w-4 flex-shrink-0" aria-hidden="true">
            <item.icon className="h-4 w-4 stroke-[2px]" />
          </span>
        )}
        <Checkbox
          checked={item.checked ?? false}
          readOnly
          className="pointer-events-none mr-3 shrink-0 scale-90"
        />
        <span className="flex-1 truncate">{item.label}</span>
        {item.shortcut && (
          <span className="ml-auto text-text-sub text-xs font-mono tracking-wide">
            {item.shortcut}
          </span>
        )}

        {/* Submenu Content for checkbox items with children */}
        {hasChildren && (
          <DropdownMenuPrimitive.SubContent
```

```
            ref={subMenuRef}
            sideOffset={4}
            align="start"
            collisionPadding={8}
            className={cn(
              "z-50 min-w-[12rem] overflow-hidden rounded-[12px] border
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
              "bg-card/85 backdrop-blur-[12px] shadow-xl p-1.5 text-text-main",
              "data-[state=open]:animate-in data-[state=closed]:animate-out",
              "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
              "data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2",
              "data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
            )}
            style={
              subMenuPosition
                ? {
                    position: "fixed",
                    top: subMenuPosition.top,
                    left: subMenuPosition.left,
                  }
                : undefined
            }
          >
            <SubMenuContext.Provider
              value={{
                level: level + 1,
                openSubMenus,
                registerSubMenu,
                unregisterSubMenu,
                isSubMenuOpen,
              }}
            >
              {item.children?.map((child) => (
                <MenuItem
                  key={child.id}
                  item={child}
                  level={level + 1}
                  onClose={onClose}
                />
              ))}
            </SubMenuContext.Provider>
          </DropdownMenuPrimitive.SubContent>
        )}
      </DropdownMenuPrimitive.CheckboxItem>
    );
  }

  // Standard Item rendering
```

```
  return (
    <DropdownMenuPrimitive.Item
      ref={itemRef}
      key={item.id}
      disabled={item.disabled}
      onSelect={
        item.onClick
          ? () => {
              item.onClick?.();
              onClose();
            }
          : undefined
      }
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm px-3 py-2
text-sm outline-none transition-all duration-100",
        "focus:bg-brand-primary/10 focus:text-brand-primary active:scale-[0.98]",
        "data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
        item.destructive
          ? "text-destructive focus:bg-destructive/10 focus:text-destructive"
          : "text-text-main",
        level > 0 && "pl-8",
      )}
      onKeyDown={handleKeyDown}
      onMouseEnter={handleMouseEnter}
    >
      {item.icon && (
        <span className="mr-3 h-4 w-4 flex-shrink-0" aria-hidden="true">
          <item.icon className="h-4 w-4 stroke-[2px]" />
        </span>
      )}
      <span className="flex-1 truncate">{item.label}</span>
      {hasChildren && (
        <DropdownMenuPrimitive.SubTrigger
          className="flex items-center ml-auto"
          aria-label="Open submenu"
        >
          <span className="text-text-sub text-xs mr-2">▶</span>
        </DropdownMenuPrimitive.SubTrigger>
      )}
      {item.shortcut && (
        <span className="ml-auto text-text-sub text-xs font-mono tracking-wide">
          {item.shortcut}
        </span>
      )}

      {/* Submenu Content */}
      {hasChildren && (
        <DropdownMenuPrimitive.SubContent
          ref={subMenuRef}
```

```
          sideOffset={4}
          align="start"
          collisionPadding={8}
          className={cn(
            "z-50 min-w-[12rem] overflow-hidden rounded-[12px] border
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
            "bg-card/85 backdrop-blur-[12px] shadow-xl p-1.5 text-text-main",
            "data-[state=open]:animate-in data-[state=closed]:animate-out",
            "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
            "data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2",
            "data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
          )}
          style={
            subMenuPosition
              ? {
                  position: "fixed",
                  top: subMenuPosition.top,
                  left: subMenuPosition.left,
                }
              : undefined
          }
        >
          <SubMenuContext.Provider
            value={{
              level: level + 1,
              openSubMenus,
              registerSubMenu,
              unregisterSubMenu,
              isSubMenuOpen,
            }}
          >
            {item.children?.map((child) => (
              <MenuItem
                key={child.id}
                item={child}
                level={level + 1}
                onClose={onClose}
              />
            ))}
          </SubMenuContext.Provider>
        </DropdownMenuPrimitive.SubContent>
      )}
    </DropdownMenuPrimitive.Item>
  );
}

// Main PopupMenu component
export function PopupMenu({
```

```
  items,
  trigger,
  triggerType = "context-menu",
  className,
}: PopupMenuProps) {
  const [openSubMenus, setOpenSubMenus] = useState<Set<string>>(new Set());
  const [, setTriggerRef] = useState<HTMLElement | null>(null);
  const menuRef = useRef<HTMLDivElement>(null);

  const registerSubMenu = useCallback(() => {
    // Registration logic for submenu tracking
  }, []);

  const unregisterSubMenu = useCallback(() => {
    // Unregistration logic
  }, []);

  const isSubMenuOpen = useCallback(
    (id: string) => {
      return openSubMenus.has(id);
    },
    [openSubMenus],
  );

  const handleOpenChange = useCallback((open: boolean) => {
    if (!open) {
      setOpenSubMenus(new Set());
    }
  }, []);

  const handleContextMenu = useCallback(
    (event: React.MouseEvent) => {
      if (triggerType === "context-menu") {
        event.preventDefault();
        event.stopPropagation();
      }
    },
    [triggerType],
  );

  const handleKeyDown = useCallback((event: React.KeyboardEvent) => {
    // Global keyboard navigation for the menu
    if (event.key === "Escape") {
      // Close all submenus
      setOpenSubMenus(new Set());
    }
  }, []);

  const contextValue: SubMenuContextValue = {
    level: 0,
```

```
    openSubMenus,
    registerSubMenu,
    unregisterSubMenu,
    isSubMenuOpen,
  };

  const renderTrigger = () => {
    if (triggerType === "context-menu") {
      return (
        <DropdownMenuPrimitive.Trigger asChild>
          <div
            ref={setTriggerRef}
            className={cn("relative", className)}
            onContextMenu={handleContextMenu}
            onKeyDown={handleKeyDown}
          >
            {trigger}
          </div>
        </DropdownMenuPrimitive.Trigger>
      );
    }

    return (
      <DropdownMenuPrimitive.Trigger asChild>
        <div
          ref={setTriggerRef}
          className={cn("relative", className)}
          onKeyDown={handleKeyDown}
        >
          {trigger}
        </div>
      </DropdownMenuPrimitive.Trigger>
    );
  };

  return (
    <DropdownMenuPrimitive.Root onOpenChange={handleOpenChange}>
      <SubMenuContext.Provider value={contextValue}>
        {renderTrigger()}

        <DropdownMenuPrimitive.Portal>
          <DropdownMenuPrimitive.Content
            ref={menuRef}
            side="bottom"
            sideOffset={4}
            align="start"
            collisionPadding={8}
            className={cn(
              "z-50 min-w-[12rem] overflow-hidden rounded-[12px] border
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
```

```
              "bg-card/85 backdrop-blur-[12px] shadow-xl p-1.5 text-text-main",
              "data-[state=open]:animate-in data-[state=closed]:animate-out",
              "data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
              "data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2",
              "data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
            )}
          >
            {items.map((item) => (
              <MenuItem
                key={item.id}
                item={item}
                level={0}
                onClose={() => handleOpenChange(false)}
              />
            ))}
          </DropdownMenuPrimitive.Content>
        </DropdownMenuPrimitive.Portal>
      </SubMenuContext.Provider>
    </DropdownMenuPrimitive.Root>
  );
}
```